CHAPTER 39

Resources, Tools and Prompts

In this chapter

Introduction: The Schema That Was a Tool . . . 3
39.1 Core Concepts . . . 4
39.2 Who Controls What . . . 5
39.3 Resources . . . 6
39.4 The Three Annotations . . . 7
39.5 Prompts . . . 7
39.6 Tools at the Protocol Layer . . . 8
39.7 Two Error Channels . . . 8
39.8 Pagination . . . 9
39.9 What a Connector Host Cannot See . . . 9
39.10 Analogies, and Where They Break . . . 10
39.11 Architecture . . . 10
39.12 Worked Example: Placing One Piece of Functionality . . . 11
39.13 Code Walkthrough: A Reader That Cannot Be Right Once . . . 12
39.14 Mini Project: The Rest of the Server . . . 13
39.15 The Whole Surface, Counted . . . 13
39.16 Reachable Surface, by Host Role . . . 14
39.17 Pagination: The Catalogue You Did Not See . . . 14
39.18 Eighteen Results, Three Readers . . . 15
39.19 outputSchema: A MUST on One Side, a SHOULD on the Other . . . 15
39.20 Annotations Are a Claim, Not a Control . . . 16
39.21 The Annotations You Can Use . . . 17
39.22 Hands-on Labs . . . 17
39.23 Lab Solutions . . . 18
39.24 Best Practices . . . 18
39.25 Common Mistakes . . . 19
39.26 Performance Considerations . . . 20
39.27 Security Considerations . . . 21
39.28 Real Production Examples . . . 21

39.29 Certification Notes 22
39.30 Twenty Practice Questions 23
39.31 Answers 23
39.32 Ten Practical Exercises 24
39.33 Five Challenge Labs 25
39.34 Ten Interview Questions 25
39.35 Summary 25
39.36 Glossary 27
39.37 Further Reading 28

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Name the three server primitives — **resources**, **tools** and **prompts** — and state which actor controls each one.
2. Choose the right primitive for a piece of functionality, and recognise the cost of choosing wrongly.
3. Work with resources: URIs, `resources/list` and `resources/read`, RFC 6570 templates, `listChanged` and per-resource subscriptions.
4. Use the three resource annotations — `audience`, `priority` and `lastModified` — to decide what enters the context window.
5. Expose and consume prompts as user-controlled entry points, including arguments and embedded resources.
6. Read a tool result correctly across **both** error channels: a JSON-RPC protocol error and an `isError: true` execution error.
7. Declare and validate an `outputSchema`, and say what `structuredContent` is and what it is not.
8. Paginate every list operation with an opaque cursor, and explain why a client must never assume a page size.

NOTE

Prerequisites

Chapter 23 for JSON Schema and constrained decoding, which `structuredContent` is explicitly *not*. **Chapter 28** for the three-reader table over stop reasons, whose shape this chapter reuses for tool results. **Chapter 32** for the declared-versus-enforced audit. **Chapter 34** for the error contract, for `is_error` as a channel addressed to the model, and for authorising at the call site rather than from the request. **Chapter 36** for the connector's tools-only limit and the three-server catalogue. **Chapter 37** for the four host roles. **Chapter 38** for the transport these messages travel on.

Deferred by name: **building, securing and shipping a server**, including the authorization framework and the completion utility, is **Chapter 40**; **MCP inside Claude Code**, where resources appear as `@` mentions and prompts as slash commands, is **Chapter 41**. **Retrieval and chunking over a large resource set** is **Chapter 53**; **prompt injection through resource content** is **Chapter 58**.

Introduction: The Schema That Was a Tool

An engineering team builds an MCP server for their data warehouse. Among its tools is `get_schema`, which returns the table definitions — about 30,000 tokens of DDL. It is genuinely useful. The model calls it, understands the tables, and writes better queries.

Then the invoice arrives. `get_schema` is being called on roughly two thirds of all turns, including turns that never touch the warehouse, because a model with a tool called `get_schema` in front of it will use it whenever the conversation drifts anywhere near a database. The team adds a line to the description — *only call this when you need the table structure* — and the rate falls, and then drifts back up as the conversation gets longer.

The problem is not the description. It is that the schema is not a tool.

MCP defines three server primitives, and the specification is unusually direct about the difference between them. Tools are **model-controlled**: *"the language model can discover and invoke tools automatically."* Resources are **application-driven**: *"host applications determining how to incorporate context based on their needs."* Prompts are **user-controlled**: *"exposed from servers to clients with the intention of the user being able to explicitly select them."*

Three primitives, three different actors deciding when each is used. A database schema is context the application should decide about — pin it, or don't. Exposing it as a tool hands that decision to the model, and the model will make it every time the topic looks close enough.

That is the spine of this chapter, and the rest of it is what follows once you take the distinction seriously. §39.16 counts what three plausible MCP servers expose across all three primitives — **72 items**, of which only 55 are tools — and finds that a connector host reaches **76.4 per cent** of that surface and cannot see the rest at all. §39.18 puts eighteen tool results through three readers and finds that the obvious one, which checks the JSON-RPC envelope, reads **every one of six execution errors as a success**. §39.20 shows two effectful tools declaring themselves read-only and two declaring nothing, against a specification that says clients **MUST** consider annotations untrusted.

The sentence to carry through: **a primitive is a decision about who decides.**

39.1 Core Concepts

DEFINITION

Resource

Data a server exposes for context, identified by a URI. Listed with `resources/list`, fetched with `resources/read`. Application-driven: the host decides what goes into the context window.

DEFINITION

Resource template

A parameterised resource, published as an RFC 6570 URI template such as `git://repo/{ref}/{path}` and listed with `resources/templates/list`. Arguments may be auto-completed through the completion API, which Chapter 40 covers.

DEFINITION

Tool

A callable operation, listed with `tools/list` and invoked with `tools/call`. Model-controlled. The protocol-level tool object carries `name`, optional `title`, `description`, `inputSchema`, optional `outputSchema`, optional `annotations`, optional `icons` and optional `execution`.

DEFINITION

Prompt

A named, argument-taking template that returns a list of messages, listed with `prompts/list` and fetched with `prompts/get`. User-controlled, and typically surfaced as a slash command.

DEFINITION

`structuredContent`

A JSON object in a tool result carrying machine-readable data. The specification is explicit that it *"is server-produced result data and is unrelated to LLM 'structured outputs' (schema-constrained model generation)"* — the mechanism Chapter 23 spent a chapter on.

DEFINITION

Protocol error

A JSON-RPC `error` object: unknown tool, malformed request, server error. It replaces the result entirely.

DEFINITION

Tool execution error

A **successful** JSON-RPC result carrying `isError: true`: an API failure, an input validation failure, a business rule. The request worked; the operation did not.

DEFINITION

Annotation (resource)

Optional metadata on a resource, template or content block: `audience` (`"user"`, `"assistant"` or both), `priority` from 0.0 to 1.0, and `lastModified` as an ISO 8601 timestamp.

DEFINITION

Cursor

An opaque string marking a position in a paginated list. Clients **MUST** treat it as opaque, **MUST NOT** assume a page size, and **SHOULD** treat a missing `nextCursor` as the end.

39.2 Who Controls What

Primitive	Controlled by	Discovered with	Used with	Typical surface
Tools	the **model**	`tools/list`	`tools/call`	invoked automatically
Resources	the **application**	`resources/list`	`resources/read`	a picker, an `@` mention, a heuristic

Primitive	Controlled by	Discovered with	Used with	Typical surface
Prompts	the **user**	`prompts/list`	`prompts/get`	a slash command

The specification adds, for each one, that implementations *"are free to expose them through any interface pattern that suits their needs—the protocol itself does not mandate any specific user interaction model."* That is a statement about interfaces, not about intent. The control model is the design; the widget is up to you.

Three tests will place almost anything correctly.

Does the model need to decide when this happens? If the answer depends on the conversation — search this, create that — it is a tool.

Is this context that either is or is not relevant, independently of the turn? A schema, a runbook, a style guide, a free/busy calendar. That is a resource, and the application should decide whether to include it. The model does not need a decision procedure for reading a file it was going to be given anyway.

Is this a workflow a person would choose by name? *Review this pull request. Triage this ticket.* That is a prompt. Making it a tool means the model may start a code review nobody asked for; making it a resource means nobody can invoke it at all.

> COMMON MISTAKE
>
> **Exposing context as a tool**
>
> **Symptom.** A tool that returns a large, slowly changing document is called far more often than it is needed, and description tuning only helps for a while. **Cause.** Tools are model-controlled. You gave the model a decision it has no stable basis for making. **Fix.** Expose it as a resource and let the application decide, or as a prompt if a person should choose it. **Detection.** Count calls per conversation against conversations where the content mattered. A ratio far above one is a misclassified primitive.

39.3 Resources

A resource is identified by a URI and nothing else. `resources/list` returns `uri`, `name`, optional `title`, `description`, `mimeType`, `icons` and `size`; `resources/read` returns `contents`, each entry carrying `uri`, `mimeType` and either `text` or a base64 `blob`.

The URI scheme is a design decision the specification has opinions about. `https://` should be used *"only when the client is able to fetch and load the resource directly from the web on its own"* — if the client must come through you to read it, use another scheme even though you will fetch it over the internet yourself. `file://` identifies things that *behave* like a filesystem and *"do not need to map to an actual physical filesystem"*. `git://` is named for version control. Custom schemes are permitted and must conform to RFC 3986.

Templates publish a shape rather than an instance: `uriTemplate: "git://repo/{ref}/{path}"`, listed separately at `resources/templates/list`. They are how a server exposes an unbounded space without enumerating it, and they are the reason `resources/list` returning three items does not mean there are

only three things to read.

Subscriptions are optional and per-resource. The `resources` capability carries two independent sub-capabilities, `subscribe` and `listChanged`, and a server may support *"neither, either, or both"*. With `subscribe`, a client sends `resources/subscribe` for a URI and receives `notifications/resources/updated` when it changes — the notification carries the URI and nothing else, so the client re-reads. With `listChanged`, the server announces that the *set* of resources has changed. They answer different questions: one is *this document moved*, the other is *there is a new document*.

Errors have their own code. **Resource not found is** `-32002`, not `-32602` and not a result with an error flag — a resource read that fails is a protocol error, because there is no result to put a flag on.

39.4 The Three Annotations

Resources, templates and content blocks may carry annotations, and all three exist to help a client decide what to put in front of the model.

`audience` is an array of `"user"` and/or `"assistant"`. A `CODEOWNERS` file annotated `["user"]` is for a person to look at; feeding it to the model is not wrong, it is just not what the server meant.

`priority` runs from 0.0 to 1.0, where *"1 means 'most important' (effectively required)"* and 0 means *"entirely optional"*. It is the server's opinion about what matters, and it is the only ordering signal you get for free.

`lastModified` is an ISO 8601 timestamp, useful for sorting by recency and for deciding whether a cached copy is stale.

These are the annotations you can act on, because the specification defines their meanings. §39.21 measures what they buy: filtering nine BUILT resources to `audience: assistant` and `priority ≥ 0.8` takes the reading budget from **288,000 bytes to 190,000**, a **34.0 per cent** reduction, without a single judgement call by you.

Contrast this with tool annotations in §39.6, where the specification defines the field and not its contents.

39.5 Prompts

A prompt has a `name`, an optional `title` and `description`, optional `icons`, and an optional list of `arguments`, each with a `name`, a `description` and a `required` flag. `prompts/get` takes the name and an arguments object and returns a `description` plus `messages` — each with a `role` of `"user"` or `"assistant"` and content that may be text, an image, audio, or an **embedded resource**.

That last one is the feature worth designing around. A prompt can return a message whose content is a full resource — `{"type": "resource", "resource": {"uri": ..., "mimeType": ..., "text": ...}}` — so a single user-selected command can assemble the runbook, the ticket and the SLA policy into one turn without the model deciding anything. The user chose; the server assembled; the model reads.

Errors are three JSON-RPC codes: an invalid prompt name and missing required arguments are both `-32602`, and an internal failure is `-32603`.

KEY TAKEAWAYS

Prompts are the primitive people forget

Every workflow you have ever written as a paragraph of instructions in a system prompt, and every "helper" tool that exists to make the model do a fixed sequence, is a prompt in disguise. Making it a prompt puts the choice back where it belongs and takes it out of the model's decision surface entirely.

39.6 Tools at the Protocol Layer

Chapters 31 to 35 covered tools as they appear in the Messages API. The MCP tool object has three fields that one does not, and one warning.

`title` is a human-readable display name, separate from `name`. Names are for matching; titles are for people.

`outputSchema` is optional, and if you provide one the obligation is asymmetric: servers **MUST** provide structured results that conform to it, and clients **SHOULD** validate. §39.19 measures what that asymmetry costs.

`execution.taskSupport` indicates whether a tool supports task-augmented execution, with values `"forbidden"` (the default), `"optional"` and `"required"`.

Tool names have their own rules here, all **SHOULD**: 1 to 128 characters, case-sensitive, only ASCII letters, digits, underscore, hyphen and dot, no spaces or commas, and *"unique within a server"*. Note the scope of that last one. Unique **within a server** — which is exactly why Chapter 37's three servers could each ship a tool called `search` while every one of them conformed.

The `inputSchema` **MUST** be a valid JSON Schema object and not `null`, defaulting to JSON Schema 2020-12 when no `$schema` is present. For a tool with no parameters the specification recommends `{"type": "object", "additionalProperties": false}` over the looser `{"type": "object"}`.

And then the warning, which is the whole of §39.20: *"For trust & safety and security, clients **MUST** consider tool annotations to be untrusted unless they come from trusted servers."*

WARNING

What this book will not tell you about tool annotations

The prose tools page of revision 2025-11-25 defines `annotations` as *"Optional properties describing tool behavior"* and names the TypeScript schema as the source of truth for the field list. It does not enumerate the keys. This chapter therefore treats the individual annotation names as **unverified from the prose** and never reads one — `read_annotation()` in the mini project raises. That is Chapter 14's rule applied to a place where guessing would have been easy and would also have been beside the point: a key whose value you must treat as untrusted is not a key you can build a gate on, whatever it is called.

39.7 Two Error Channels

This is the part of the protocol that costs people most, because both channels look like success from the wrong angle.

A **protocol error** is a JSON-RPC error object. There is no `result`. It means the request itself was wrong: an unknown tool, a body that fails the `CallToolRequest` schema, or the server failing internally.

A **tool execution error** is a perfectly ordinary successful JSON-RPC result that happens to carry `isError: true`, plus content explaining what went wrong. It means the request was fine and the operation was not: an upstream 503, a date in the past, a refund outside the policy window.

The specification says why the distinction exists, and it is not bookkeeping. Execution errors *"contain actionable feedback that language models can use to self-correct and retry with adjusted parameters"*, and clients **SHOULD** pass them to the model. Protocol errors *"indicate issues with the request structure itself that models are less likely to be able to fix"*, and clients **MAY** pass them along but should expect less.

Chapter 28 built three readers over the Messages API's stop reasons and got 6/2/10, 15/3/0 and 16/0/2. §39.18 does the same thing here and the numbers are cleaner and more damning: a reader that checks the JSON-RPC envelope gets **12 of 18 correct and reads all six execution errors as successes**; a reader that checks `isError` gets 12 correct and **raises on all six protocol errors**; a reader that checks both gets **18 of 18**.

KEY TAKEAWAYS

One rule, restated for a new layer

Chapter 34's rule was that `is_error: true` is a channel addressed to the model. Chapter 39's is that it is a channel addressed to the model *inside a successful envelope*, so a client that stops reading at the envelope will report a clean success while handing the model an error string — or worse, handing it nothing.

39.8 Pagination

All four list operations paginate: `tools/list`, `resources/list`, `resources/templates/list` and `prompts/list`. The model is a cursor, and the rules are short.

The cursor is *"an opaque string token"*. **Page size is determined by the server, and clients MUST NOT assume a fixed page size.** Clients **MUST** treat cursors as opaque — do not parse them, do not modify them, do not persist them across sessions. A missing `nextCursor` means the end. An invalid cursor **SHOULD** produce `-32602`.

The failure mode is silent and total. A client that calls `tools/list` once and ignores `nextCursor` sees one page and concludes that is the catalogue. §39.17 measures it: at a server-chosen page size of 50, a one-shot client misses **5 of 55 tools** and none of the resources or prompts, because those lists are short. At a page size of 5 — which the server is entirely free to choose, and which you **MUST NOT** assume it will not — the same client misses **54 of 72 items**.

Nothing errors. The server did what it was asked. The client simply has a smaller world than the one it was offered, and every downstream conclusion — *this server can't do that* — is wrong.

39.9 What a Connector Host Cannot See

Chapter 36 verified that the MCP connector supports *"only tool calls"* of the MCP feature set. §39.16 turns that sentence into a number.

Across three plausible servers: **55 tools, 9 resources, 3 resource templates and 5 prompts — 72 items**. A connector host reaches the 55 tools, which is **76.4 per cent** of the surface. It cannot list a resource, cannot read one, cannot subscribe to one, cannot enumerate a template and cannot invoke a prompt.

That is not an argument against the connector. It is an argument for knowing what you are buying, and for two design consequences that follow directly. First, if your server is intended to be reached through the connector, anything you expose as a resource is invisible — server authors targeting connector clients end up publishing the same content as a tool, which reintroduces §39.2's problem by a different route. Second, if you are choosing a host role and your servers lead with resources and prompts, the connector is the wrong choice and the client-side SDK helpers of Chapter 37 are the right one.

39.10 Analogies, and Where They Break

VISUAL ANALOGY

The reference shelf, the toolbox and the recipe card

Resources are the reference shelf: the schema diagram, the runbook, the style guide. Somebody decides what to take down before the work starts, and that somebody is the person running the workshop.

Tools are the toolbox. Whoever is doing the work reaches for what the job needs, without asking.

Prompts are recipe cards. The customer picks one by name, and it says which tools to use and which references to fetch.

Where it breaks. A real apprentice who kept fetching the same manual would notice and stop. A model has no memory of the last conversation and no cost signal, so it will fetch the manual again on the next turn and every turn after that. The analogy also under-sells subscriptions: a shelf does not tell you when a book has changed, and `resources/subscribe` does.

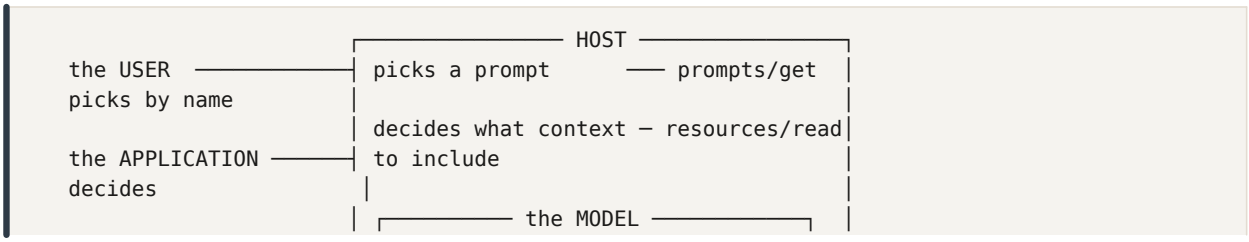
VISUAL ANALOGY

The receipt and the refusal

Two ways a shop can fail you. It can refuse to take your order at all — wrong form, unknown product, till broken — which is a protocol error, and you get no receipt. Or it can take the order, print a receipt, and have the receipt say *out of stock*. That is `isError: true`: a complete, well-formed transaction whose content is bad news.

Where it breaks. A person reading a receipt reads the words. A program that checks only whether a receipt was printed concludes everything went fine — which is exactly the 12 of 18 in §39.18.

39.11 Architecture



```
the MODEL ──────────┬┤ chooses a tool ─── tools/call   │  │
chooses per turn    │ └──────────────────────────────┘  │
                    └──────────────────────────────────┘
                                       │  one client per server
                                       ▼
                    ┌──────────────────────────────────┐
                    │ SERVER: tools · resources · prompts   │
                    └──────────────────────────────────┘
Get the primitive wrong and you have moved the decision to the wrong
actor. That is the whole of §39.2.
```

```
┌──────────────────────── the response ────────────────────────┐
│                                                             │
│  {"error": {...}}                  PROTOCOL ERROR          │
│        no result at all            unknown tool, bad body  │
│        MAY be shown to the model   -32601 / -32602 / -32603│
│                                                             │
│  {"result": {"content": [...],     EXECUTION ERROR         │
│              "isError": true}}     upstream 503, bad date  │
│        a SUCCESSFUL result         SHOULD be shown to the  │
│                                    model, to self-correct  │
│                                                             │
│  {"result": {"content": [...],     SUCCESS                 │
│              "structuredContent":  conforms to outputSchema│
│                {...}}}             if one was declared     │
└─────────────────────────────────────────────────────────────┘
  envelope-only reader: 12/18 correct, 6 execution errors read as OK
  isError-only reader:  12/18 correct, raises on all 6 protocol errors
  both channels:        18/18
```

```
tools/list  ──────────────────►  page 1 (50 tools) + nextCursor
                                          │
a PAGING client:                          ▼
tools/list {cursor} ──────────►  page 2 (5 tools), no nextCursor  = 55
a ONE-SHOT client:
(ignores nextCursor)                                              = 50
                                                 5 tools invisible
at a server-chosen page size of 5:  54 of 72 items invisible
"Page size is determined by the server, and clients MUST NOT assume
 a fixed page size."
```

39.12 Worked Example: Placing One Piece of Functionality

A team wants Claude to be able to work with their incident runbooks. Four questions, in order.

1. Who should decide when this is used? Nobody wants the model deciding to read a runbook mid-conversation about something else. The application should attach the relevant runbook when an incident is in scope. **Resource.**

2. What URI scheme? Not `https://`, because the client cannot fetch these directly — it must come through the server for authorisation. A custom scheme, `runbook://service/{name}`, conforming to RFC 3986. And because the set is large and shaped, it is a **template** listed at `resources/templates/list`, not thirty enumerated resources.

3. What annotations? `audience: ["assistant"]`, since these are for the model to read. `priority` reflecting how often each one matters. `lastModified` from the document store, so the client can sort by

recency and invalidate a cache.

4. Is there a workflow on top? Yes: *triage this incident against the runbook*. That is a person's decision, made by name, so it becomes a **prompt** — `triage_incident`, taking an incident key, returning messages that embed the runbook resource and the incident detail. The user chooses; the server assembles; the model reads.

The tool in this design is the small thing at the end: `update_incident_status`, which does something to the world and which the model should invoke when the conversation calls for it. One tool, one template, one prompt, and a system in which nobody is deciding anything they should not be.

39.13 Code Walkthrough: A Reader That Cannot Be Right Once

Two modules. The first extends Chapter 36's three servers with the two primitives they were missing; the second reads results.

The reader trio is the heart of it, and the interesting thing is how little separates them:

```
def read_jsonrpc_only(payload):
    """Checks the JSON-RPC envelope and stops."""
    if "error" in payload:
        return "protocol-error"
    return "success"
def read_iserror_only(payload):
    """Checks `isError` and stops."""
    result = payload.get("result")
    if result is None:
        raise KeyError("no 'result' in payload")
    return "execution-error" if result.get("isError") else "success"
def read_both(payload):
    """The only reader that can be right twice."""
    if "error" in payload:
        return "protocol-error"
    result = payload["result"]
    return "execution-error" if result.get("isError") else "success"
```

The first reader is what you write if you think in transports — Chapter 38's world, where an envelope is the unit. It is correct about every protocol error and it is structurally incapable of noticing that a result said `isError: true`. The second is what you write if you think in applications. It is correct about every execution error and it dereferences a `result` that is not there.

`read_both` is four lines longer than either and it is the only one that is ever right about a failure it did not expect. That is the whole argument.

The conformance checker is deliberately partial, and the comment says why:

```
def conforms(value, schema):
    """Enough of JSON Schema 2020-12 to check the documented MUST.
    `additionalProperties` is absent from the published example, so an
    undeclared field is permitted - which is the behaviour a client
    that assumes closed objects will get wrong.
    """
```

Chapter 23 established that `additionalProperties` must be present and `false` for the Messages API's structured outputs. Nothing of the kind applies here: `outputSchema` is ordinary JSON Schema, the published example omits `additionalProperties` entirely, and a server that adds a field is conforming. A client written on Chapter 23's habits will reject a valid result.

And one guard that exists to refuse rather than to compute:

```
def read_annotation(tool_name, key):
    """Refuse to read a key the prose page never defined."""
    if key not in ANNOTATION_KEYS_VERIFIED_FROM_PROSE:
        raise Unverified(...)
```

`ANNOTATION_KEYS_VERIFIED_FROM_PROSE` is the empty tuple. The prose tools page defines `annotations` as *"Optional properties describing tool behavior"* and points at the TypeScript schema for the field list, so this book has verified no key name from it, and the function raises on every call. It is a strange-looking piece of code and it is the honest one: a value you **MUST** treat as untrusted is not a value to build a gate on, and pretending otherwise would have been the easiest mistake in the chapter.

39.14 Mini Project: The Rest of the Server

What it is. Chapter 36's three servers, extended with the resources, templates and prompts they would really have; a reachable-surface audit per host role; a pagination harness; the three-reader table over eighteen tool results; an `outputSchema` conformance check; and an annotation-trust comparison in Chapter 34's call-site shape.

```
   primitives.py ──┬─▶ RESOURCES · TEMPLATES · PROMPTS   (BUILT)
                   ├─▶ surface() / total_surface()
                   ├─▶ reachable_surface(role)  ── Ch 36's tools-only limit
                   ├─▶ paginate() / list_all(follow_cursor=?)
                   └─▶ annotation_budget(audience, priority_floor)
                              │
                              └──▶ mcpcat.py  (Ch 36's 55 tools)
   results.py ─────┬─▶ 18 fixtures: 6 success, 6 execution, 6 protocol
                   ├─▶ read_jsonrpc_only / read_iserror_only / read_both
                   ├─▶ conforms()  outputSchema, additionalProperties absent
                   ├─▶ annotation_audit()  trusted gate vs derived gate
                   └─▶ read_annotation()   raises: no key is verified
   report.py ──────────▶ every figure below   test_ch39.py ──▶ 27 tests
```

Folder structure. `ch39/` carries forward Chapter 38's set — `stdioframe.py`, `httpwire.py`, `resume.py`, `lifecycle.py`, `roles.py`, `blast.py`, `hostmatrix.py`, `mcpcat.py` and the rest — and adds `primitives.py`, `results.py`, `report.py` and `test_ch39.py`.

Deployment and evaluation. `python3 report.py` prints everything in about a second; `python3 -m pytest -q` runs **27 tests**, all passing from a clean restore.

Provenance. Every protocol rule is `DOC` from specification revision 2025-11-25, read 5 August 2026. The nine resources, three templates, five prompts, eighteen result fixtures and eight annotation claims are `BUILT`.

39.15 The Whole Surface, Counted

```
server          tools  resources  templates  prompts
code-mcp           32          4          1        2
calendar-mcp       14          2          0        1
tickets-mcp         9          3          2        2
ALL                55          9          3        5   total 72
```

Chapter 36 audited 55 tools and drew conclusions about a catalogue. This is the same three servers with the primitives Part 6 had no vocabulary for: **72 items, and 17 of them are not tools.**

Notice `calendar-mcp` has no templates and `code-mcp` has one. Templates are the primitive that scales: `git://repo/{ref}/{path}` is one published item standing for every file at every revision, which is why counting `resources/list` tells you almost nothing about how much a server can show you.

39.16 Reachable Surface, by Host Role

```
connector       55 of  72  (76.4%)   unreachable: {'resources': 9,
                                      'resource_templates': 3, 'prompts': 5}
sdk_helpers     72 of  72  (100.0%)  unreachable: {}
claude_code     72 of  72  (100.0%)  unreachable: {}
```

76.4 per cent. The connector reaches every tool and none of the rest, exactly as Chapter 36 recorded and Chapter 37 tabulated, and this is what the sentence costs on a realistic server.

Two consequences follow, and both are design decisions rather than complaints.

If you are **writing** a server for connector clients, your resources are invisible. The pressure will be to republish them as tools — `get_schema`, `get_runbook`, `get_sla_policy` — which is precisely §39.2's failure reintroduced through the back door. The better answer is usually to keep the resource and add one narrow tool that returns *the specific thing* rather than the whole document, so the model is deciding about a fact rather than about a file.

If you are **choosing** a host role and the servers you care about lead with resources and prompts, the connector is the wrong instrument. Chapter 37's client-side SDK helpers reach all three primitives because they run a real MCP client in your process.

39.17 Pagination: The Catalogue You Did Not See

```
operation                     available   paging  one-shot  invisible
tools/list                           55       55        50          5
resources/list                        9        9         9          0
resources/templates/list              3        3         3          0
prompts/list                          5        5         5          0
at a server-chosen page size of 5, invisible items: 54 of 72
```

At a page size of 50 the damage is five tools. That is small enough to be invisible in testing and large enough to matter: five tools the model was never told about, and five capabilities a user will be told the assistant does not have.

The second line is the one to take seriously. Page size *"is determined by the server"* and clients **MUST NOT** assume a fixed one. A server that chooses 5 — because its lists are usually short, because it is under load, because someone changed a default — turns the same one-shot client into one that sees **18 of 72 items**. Nothing errors, on either side. The server offered a `nextCursor` and the client did not ask.

Three rules make this safe, and all three are in the specification. Follow `nextCursor` until it is absent. Treat the cursor as opaque — do not parse it, do not modify it, and **do not persist it across sessions**, which is the one people get wrong when they try to be clever about caching. And handle `-32602` on an invalid cursor by restarting the list rather than by failing.

39.18 Eighteen Results, Three Readers

Chapter 28 built three readers over the Messages API's stop reasons and scored them 6/2/10, 15/3/0 and 16/0/2 over eighteen fixtures. Here are three readers over eighteen MCP tool results — six successes, six execution errors, six protocol errors.

```
jsonrpc envelope only    correct 12  misread  6  raised  0  of 18
isError only             correct 12  misread  0  raised  6  of 18
both channels            correct 18  misread  0  raised  0  of 18
   jsonrpc envelope only misreads e.g. date in the past
   (execution-error read as success), upstream 503
   (execution-error read as success)
```

The envelope reader misreads six and raises none. Every execution error is reported as a success. The upstream is down, the date is invalid, the refund is outside policy — and the client's log says the tool call succeeded. This is the failure mode Chapter 25 named for refusals and Chapter 28 named for stop reasons, arriving for the third time: *a failure that is delivered inside a successful envelope will not be seen by anything that watches envelopes.*

The `isError` reader misreads none and raises six. It is wrong in the loud direction. Chapter 28's rule stands: a reader that raises has told you something; one that returns a wrong answer has not.

Both channels: 18 of 18. Four extra lines.

And there is a consequence beyond monitoring. The specification says clients **SHOULD** provide execution errors to the model, because they *"contain actionable feedback that language models can use to self-correct and retry with adjusted parameters"*. A client that cannot distinguish the two channels either passes protocol errors to the model — noise the model cannot act on — or drops execution errors, which is worse: the model is told the call succeeded, and it will reason from a result that says `Invalid departure date: must be in the future.` as though it were data.

39.19 `outputSchema`: A MUST on One Side, a SHOULD on the Other

```
structured results                          6
accepted by a client that does not validate 6
accepted by a validating client             2
checker agrees with the hand labels         6 of 6
disagreements: none
```

Six structured results against one declared `outputSchema`. Servers **MUST** conform; clients **SHOULD** validate. Four of the six do not conform — a missing required field, a number sent as a string, a `null` where a string was declared, and an empty object — and a client that skips validation accepts **all six**.

That is the asymmetry in one line. The obligation is on the server, the enforcement is optional and on the client, and nothing in between checks. It is Chapter 32's finding in a new place: declared is not enforced, and the gap is exactly as wide as your own validator.

The fourth result is the interesting one. `{"temperature": 22.5, "conditions": "Partly cloudy", "humidity": 65, "wind": 3.1}` carries an undeclared field and it **conforms**, because `additionalProperties` is absent from the schema and JSON Schema's default is permissive. Chapter 23 trained a habit here — the Messages API's structured outputs require `additionalProperties` to be present and `false` — and that habit is wrong for `outputSchema`, which is ordinary JSON Schema with ordinary defaults. A client that rejects the fourth result is rejecting a valid one.

> KEY TAKEAWAYS
>
> **Two things called structured, one relationship between them: none**
>
> Chapter 23's structured outputs constrain what the **model** generates, by editing the token distribution at sampling time. `structuredContent` is what a **server** returns, validated afterwards or not at all. The specification says so explicitly, and the two features share nothing but an adjective.

39.20 Annotations Are a Claim, Not a Control

```
tools examined                              8
passed a gate that TRUSTS the annotations   3
passed a gate DERIVED from the catalogue    1
effectful tools claiming to be read-only    ['refund_order',
                                             'share_calendar_publicly']
effectful tools claiming nothing at all     ['delete_all_events',
                                             'transition']
annotation keys verified from the prose page NONE
reading one raises: the annotation key 'readOnlyHint' is not defined on
   the prose tools page of revision 2025-11-25
```

Eight tools, and two gates over them. One trusts what the server says about itself; the other derives the answer from the catalogue with Chapter 36's effect cues, which is Chapter 34's rule — *the validator must be derived from the audit* — applied one layer out.

The trusting gate passes **three** tools as safe. Two of them refund money and share a calendar publicly. The derived gate passes **one**, and it is `search`.

Then the quieter half. Two effectful tools declare **nothing at all** — `delete_all_events` and `transition` — and a gate that reads annotations has to decide what an absent annotation means. Absent is not "safe" and it is not "dangerous"; it is silence, and silence from an untrusted party carries no information in either direction. A gate keyed on annotations therefore needs a default, and every default it could pick is wrong for half the catalogue.

The specification's own instruction is the short version: clients **MUST** consider tool annotations untrusted unless they come from trusted servers. And "trusted server" is doing real work in that

sentence — it means a server whose code you or your organisation controls, which is exactly the case where you did not need the annotation, because you already knew.

COMMON MISTAKE

Gating destructive operations on a server's own annotations

Symptom. A confirmation prompt appears for some destructive tools and not others, with no pattern anyone can explain. **Cause.** The pattern is which server author remembered to annotate honestly. **Fix.** Derive the classification at your call site — from your own audit of the schema, the name and the documented behaviour — and treat annotations as a display hint. **Detection.** Run both gates over your catalogue and diff them. Here: 3 against 1.

39.21 The Annotations You Can Use

Resource annotations are the opposite case: the specification defines what each one means, so acting on them is reasoning rather than trusting.

```
audience=assistant, no floor       8 resources, 288000 bytes
audience=assistant, priority>=0.8  5 resources, 190000 bytes
reduction 34.0%
```

One filter on `audience` removes the resource annotated `["user"]` — the `CODEOWNERS` file, which was never meant for the model. A `priority` floor of 0.8 removes three more, and the reading budget falls by **34.0 per cent** with no judgement from you at all.

This still costs you something if the server's priorities are wrong, and that is the honest caveat: `priority` is the server's opinion. But it is an opinion the specification gives a defined meaning — 1 is *"effectively required"*, 0 is *"entirely optional"* — which is precisely what tool annotations lack. You are acting on a documented scale, not on an undocumented key.

`lastModified` is the third, and it does the job Chapter 30's cache diagnostics did in another register: it lets you decide whether to re-read at all.

39.22 Hands-on Labs

Lab 1 — Reclassify one thing. Find a tool in your own catalogue that returns a large, slowly changing document. *Success criterion:* you can say which primitive it should be and what the application or the user would do instead of the model.

Lab 2 — Break a one-shot client. Set the page size in `pagination_audit` to 3 and predict the invisible count before running it. *Success criterion:* your prediction matches, and you can state the rule that makes a fixed-page-size assumption illegal.

Lab 3 — Score your own reader. Take the reader in your MCP client and run it over the eighteen fixtures. *Success criterion:* a correct/misread/raised triple, and a diff against `read_both`.

Lab 4 — Validate an `outputSchema`. Pick a server that declares one, capture ten results, and validate them. *Success criterion:* a conformance rate, and an explicit decision about undeclared fields.

Lab 5 — Diff the two gates. Implement both the trusting and the derived gate over your real catalogue. *Success criterion:* the number of tools where they disagree, and a written decision about which one runs in production.

39.23 Lab Solutions

Lab 1. The test in §39.2 is the fastest route: does the model need to *decide* when this happens? A schema, a policy document, a runbook and a style guide all answer no. The replacement is usually a resource plus one narrow tool — keep the document as a resource and expose `lookup_table(name)` rather than `get_schema()`, so what the model decides about is a fact rather than a file.

Lab 2. At page size 3, every list overflows: 55 tools, 9 resources, 3 templates and 5 prompts become 3, 3, 3 and 3, so a one-shot client sees 12 of 72 and misses 60. The rule is *"Page size is determined by the server, and clients MUST NOT assume a fixed page size"* — the client cannot even conclude that a full-looking page is full.

Lab 3. Most hand-written readers are `read_jsonrpc_only` with extra logging. The tell is a `try`/`except` around JSON-RPC errors and no mention of `isError` anywhere. Score 12/6/0 and the fix is four lines.

Lab 4. The decision about undeclared fields is the substance. `additionalProperties` is absent from the specification's own example, so the permissive reading is correct, and a client that rejects extra fields will reject conforming results the day a server adds one. If you want closed objects, that is a conversation with the server author, not a client-side assumption.

Lab 5. Disagreements are the finding. Every tool where the trusting gate says "safe" and the derived gate says "effectful" is a place your confirmation prompt would not have appeared. Here that set is `{refund_order, share_calendar_publicly}` — two of the three tools the trusting gate waved through.

39.24 Best Practices

BEST PRACTICE

Choose the primitive by asking who decides

Model decides per turn → tool. Application decides per session → resource. User decides by name → prompt. This one question prevents most of the failures in this chapter.

BEST PRACTICE

Publish templates, not enumerations

`git://repo/{ref}/{path}` at `resources/templates/list` beats a thousand entries at `resources/list`, and it is the only way to expose an unbounded space honestly.

BEST PRACTICE

Annotate resources with `audience`, `priority` and `lastModified`

They have defined meanings, so a client can act on them. Here they cut a reading budget by 34 per cent with no judgement calls.

BEST PRACTICE

Read both error channels, always

Four lines. 12 of 18 becomes 18 of 18, and execution errors reach the model where they can do some good.

BEST PRACTICE

Pass execution errors to the model and think twice about protocol errors

The specification's own asymmetry: SHOULD for the first, MAY for the second, because only the first is something a model can act on.

BEST PRACTICE

Declare an `outputSchema` and validate it on the client

The server's obligation is a MUST you cannot enforce. Your validator is the only thing that checks.

BEST PRACTICE

Follow `nextCursor` everywhere, and never persist a cursor

All four list operations paginate. Treat cursors as opaque, restart on `-32602`, and do not carry them across sessions.

BEST PRACTICE

Treat tool annotations as display metadata

Derive your safety classification at the call site. Annotations are untrusted by specification, and silence is not safety.

39.25 Common Mistakes

COMMON MISTAKE

Making context a tool

Symptom. A document-returning tool is called on most turns regardless of relevance. **Cause.** Tools are model-controlled. **Fix.** Resource, or a narrow tool returning a fact instead of a file. **Detection.** Calls per conversation against conversations where the content was used.

COMMON MISTAKE

Checking the JSON-RPC envelope and stopping

Symptom. A dashboard shows a 100 per cent tool success rate while users report failures. **Cause.** `isError: true` lives inside a successful result. **Fix.** `read_both`. **Detection.** 12 of 18 in $39.18.

COMMON MISTAKE

Dereferencing `result` without checking for `error`

Symptom. A `KeyError` or a null dereference on every protocol error. **Cause.** A protocol error has no `result` at all. **Fix.** Check for `error` first. **Detection.** Six raises out of eighteen.

COMMON MISTAKE

Calling a list operation once

Symptom. The assistant says it cannot do something the server clearly offers. **Cause.** `nextCursor` was ignored. **Fix.** Loop until it is absent. **Detection.** Compare your catalogue size against the server's documentation. Five missing tools at page size 50; fifty-four items at page size 5.

COMMON MISTAKE

Applying Chapter 23's `additionalProperties` rule to `outputSchema`

Symptom. Valid results are rejected when a server adds a field. **Cause.** `outputSchema` is ordinary JSON Schema; the constrained-decoding subset does not apply. **Fix.** Permit undeclared fields unless the schema says otherwise. **Detection.** Any rejection whose only cause is an extra key.

COMMON MISTAKE

Gating on tool annotations

Symptom. Destructive operations skip confirmation, inconsistently. **Cause.** Annotations are the server's claim about itself and are untrusted by specification. **Fix.** Derive the classification from your own audit. **Detection.** 3 tools pass a trusting gate, 1 passes a derived one.

COMMON MISTAKE

Treating an absent annotation as a safe default

Symptom. `delete_all_events` runs without confirmation because it declared nothing. **Cause.** Silence carries no information, and a default has to be chosen. **Fix.** Default to "unknown, therefore confirm", and derive the real answer elsewhere. **Detection.** Two of eight tools here declare nothing at all.

39.26 Performance Considerations

Resources are the cheapest primitive and the easiest to overspend. A tool call costs a round trip; a resource read costs a round trip and then sits in the context window for the rest of the conversation. Chapter 20's ordering rule applies directly: a resource read once and kept is stable prefix material and

caches well; a resource re-read every turn is variable content at the wrong end of the prompt.

Subscriptions beat polling, when they exist. `resources/subscribe` gives you a notification when a specific URI changes. Without it, the only options are re-reading on a timer — mean staleness of half the interval, and a token cost every time — or accepting staleness. Check the `subscribe` sub-capability before you design around it; a server may support neither, either or both.

Pagination has a latency cost you can measure. A 55-tool catalogue at page size 5 is eleven round trips before the first user turn. Page size is the server's to choose, so the client's job is to page concurrently where the transport allows it and to cache the assembled catalogue for the session — not the cursors, which must not be persisted.

`structuredContent` **costs tokens twice.** The specification says a tool returning structured content **SHOULD** also return the serialised JSON in a text block for backwards compatibility. That is deliberate duplication, and on a large result it doubles the token bill. If every client you support reads `structuredContent`, the text block is a cost with no reader.

39.27 Security Considerations

Resource content is untrusted input. Chapter 31 established the rule for tool results and it applies unchanged to `resources/read`: the bytes came from somewhere else, they may contain instructions addressed to the model, and they belong inside a result block rather than promoted into a system prompt. A resource is more dangerous than a tool result in one specific way — it is often *large*, *pinned* and *re-sent every turn*, so a single injected line is present for the whole conversation. Chapter 58 takes this properly.

The server's own obligations. Servers **MUST** validate all resource URIs, **SHOULD** implement access controls on sensitive resources, **MUST** properly encode binary data, and **SHOULD** check permissions before operations. For tools, servers **MUST** validate all inputs, implement access controls, rate limit invocations and sanitise outputs.

The client's own obligations. Clients **SHOULD** prompt for confirmation on sensitive operations, **show tool inputs to the user before calling the server** — the specification's stated reason is *"to avoid malicious or accidental data exfiltration"*, which is the outbound half of Chapter 37's argument about tool arguments as a disclosure — validate results before passing them to the model, implement timeouts, and log usage for audit.

Prompt inputs and outputs. The prompts page is brief and blunt: implementations **MUST** carefully validate all prompt inputs and outputs *"to prevent injection attacks or unauthorized access to resources"*. A prompt that embeds resources is assembling a turn from several sources at once, and each of them is a place an instruction could hide.

Annotations, one more time. Clients **MUST** consider tool annotations untrusted unless they come from trusted servers. §39.20 is what that sentence is worth in practice.

39.28 Real Production Examples

A data platform that moved its schema. The `get_schema` tool became a `warehouse://schema/{database}` resource template plus a narrow `describe_table(name)` tool. Schema tokens went from being fetched on most turns to being attached by the application when a warehouse question was in scope, and the narrow tool answered the cases where the model genuinely needed one table's shape.

A support desk that reads both channels. After a month of a green dashboard and unhappy users, the team found their client checked only the JSON-RPC envelope. The fix was four lines. The visible change was that their tool success rate dropped from 100 per cent to 94 — which was the true number all along, and which is now something they can act on.

A server author who paginates at 20. Their tool list is 40 items. They chose 20 deliberately, in staging, precisely to catch clients that do not follow `nextCursor` before those clients reach production. Three integrations broke in the first week, all of them one-shot readers, all of them fixed before launch.

39.29 Certification Notes

Concept	Exam domain	Weight
The three primitives and their controllers	D2 Tool Design & MCP Integration	18%
Choosing a primitive for a piece of functionality	D2 · D1 Agentic Architecture	18% · 27%
Resource URIs, templates and subscriptions	D2 · D5 Context Management & Reliability	18% · 15%
Resource annotations and context budgeting	D5	15%
Prompts as user-controlled entry points	D2 · D4 Prompt Engineering	18% · 20%
Protocol errors versus execution errors	D5 · D2	15% · 18%
`outputSchema` and `structuredContent`	D4 · D2	20% · 18%
Cursor pagination and the page-size rule	D2	18%
Tool annotations as untrusted	D5 · D2	15% · 18%

CERTIFICATION NOTE

Three reliable question shapes

Placement. A scenario describing a large, rarely changing document behind a tool is asking you to name *resource*. A scenario describing a named workflow a user picks is asking you to name *prompt*. The distractor is always "improve the tool description".

Error channels. Any question containing `isError` is testing whether you know it lives inside a **successful** result. `isError: true` is not a JSON-RPC error and does not have an error code.

Untrusted metadata. Questions about tool annotations have exactly one correct answer shape: clients MUST treat them as untrusted, so the safety decision belongs at your call site. Options that say "check the annotation before calling" are wrong however sensible they sound.

39.30 Twenty Practice Questions

1. Name the three server primitives and the actor that controls each.
2. Which primitive is *model-controlled*?
3. Which primitive is typically surfaced as a slash command?
4. What identifies a resource?
5. What is a resource template, and which method lists them?
6. Which two sub-capabilities may a `resources` capability declare, and must a server support both?
7. What JSON-RPC code does a server return for a resource that does not exist?
8. Name the three resource annotations and say what `priority: 1` means.
9. What may a `PromptMessage` contain besides text?
10. Which four list operations paginate?
11. May a client assume a page size? What must it treat cursors as?
12. What should a client do when `nextCursor` is absent?
13. Give two examples of a protocol error and two of a tool execution error.
14. Is a result with `isError: true` a JSON-RPC error?
15. Which kind of error should a client pass to the model, and why?
16. If a tool declares an `outputSchema`, what must the server do and what should the client do?
17. Is `structuredContent` related to constrained decoding?
18. Does an undeclared field in `structuredContent` violate the published example schema?
19. What must a client assume about tool annotations?
20. What does the specification recommend as the `inputSchema` for a tool with no parameters?

39.31 Answers

1. **Resources** (application-driven), **tools** (model-controlled), **prompts** (user-controlled).
2. **Tools.** The distractor is prompts, because both end up as text the model sees.
3. **Prompts.** The specification's own example is a slash command.

4. **A URI.** Not a name — `name` exists but is not the identifier.
5. A parameterised resource published as an **RFC 6570 URI template**, listed with `resources/templates/list`. It is a separate method; templates do not appear in `resources/list`.
6. `subscribe` and `listChanged`, and **no** — a server may support *"neither, either, or both"*.
7. `-32002`**.** Distractors offering `-32602` are describing an invalid-params error, which is what a bad prompt name or a bad cursor returns.
8. `audience`, `priority`, `lastModified`. `priority: 1` means *"most important"*, described as *"effectively required"*.
9. **Image content, audio content, and an embedded resource.** The embedded resource is the one worth remembering.
10. `tools/list`**,** `resources/list`**,** `resources/templates/list`**,** `prompts/list`**.**
11. **No** — page size is the server's and clients **MUST NOT** assume a fixed one. Cursors must be treated as **opaque**: not parsed, not modified, not persisted across sessions.
12. **Treat it as the end of the results.**
13. Protocol: an **unknown tool**, a **malformed request**, a server error. Execution: an **upstream API failure**, an **input validation failure** such as a date in the past, a **business rule** such as a refund outside policy.
14. **No.** It is a *successful* result whose content reports a failure. There is no error code.
15. **Tool execution errors** — clients **SHOULD** provide them, because they contain actionable feedback the model can use to self-correct. Protocol errors **MAY** be provided but are *"less likely to result in successful recovery"*.
16. Servers **MUST** provide structured results conforming to it; clients **SHOULD** validate. The asymmetry is the point.
17. **No.** The specification states it explicitly: `structuredContent` is server-produced result data and is *"unrelated to LLM 'structured outputs'"*.
18. **No.** `additionalProperties` is absent from the published example, so the default permissive behaviour applies. Chapter 23's rule about `additionalProperties: false` belongs to the Messages API's constrained decoding, not here.
19. That they are **untrusted**, unless the server is one you trust. The safety decision belongs at your call site.
20. `{"type": "object", "additionalProperties": false}`, described as the recommended option because it explicitly accepts only empty objects. `{"type": "object"}` is valid but looser; `null` is invalid.

39.32 Ten Practical Exercises

1. Classify every tool in your catalogue as tool, resource or prompt using §39.2's three questions.
2. Count how many of your tools return documents rather than facts.
3. Write one resource template for a space you currently enumerate.
4. Add `audience`, `priority` and `lastModified` to your resources and compute the budget reduction.

5. Check whether each of your servers declares `subscribe`, `listChanged`, both or neither.
6. Run your client's result reader over the eighteen fixtures and score it.
7. Find every place your code touches `result` without checking for `error`.
8. Add `nextCursor` following to any list call that lacks it.
9. Declare an `outputSchema` on one tool and validate it client-side.
10. Diff a trusting gate against a derived gate over your real catalogue.

39.33 Five Challenge Labs

1. **A resource budgeter.** Given annotations and a token budget, choose which resources to include, in Chapter 15's cost-per-outcome shape rather than by priority alone.
2. **A subscription-aware cache.** Implement `resources/subscribe` handling with a fallback to timed re-reads, and measure mean staleness for both against a synthetic change stream.
3. **A prompt that assembles.** Build a prompt returning embedded resources, and measure the token cost against the equivalent sequence of tool calls.
4. **A conformance monitor.** Validate every `structuredContent` against its `outputSchema` in production, and report the per-server conformance rate as an operational metric.
5. **A derived safety classifier.** Extend Chapter 36's effect cues into a real classifier over your catalogue, with Wilson intervals in Chapter 8's shape, and publish where it is blind.

39.34 Ten Interview Questions

1. Why does MCP have three primitives instead of one?
2. When is something a resource rather than a tool, and what goes wrong if you get it backwards?
3. What is a resource template for?
4. Why does `resources/read` failing produce a protocol error rather than an `isError` result?
5. Explain the two error channels and why the specification treats them differently.
6. What is `structuredContent`, and how does it relate to constrained decoding?
7. Why can a client not assume a page size?
8. What would you do with a tool annotation that says a tool is read-only?
9. How would you decide which resources go into a limited context budget?
10. A connector-hosted assistant cannot see a server's resources. What are your options?

39.35 Summary

MCP's server side is three primitives, and the specification names a different controller for each: tools are **model-controlled**, resources are **application-driven**, and prompts are **user-controlled**. That is not a taxonomy, it is an allocation of decisions, and most of the trouble in this chapter comes from putting a decision in the wrong place. A database schema behind a tool is a decision the model has no stable basis for making, and no amount of description tuning turns it into one; as a resource the application decides,

and as a prompt a person does. The three questions in §39.2 — does the model need to decide when, is this context independent of the turn, would a person pick this by name — place almost anything correctly.

Counted properly, three plausible servers expose **72 items**: 55 tools, 9 resources, 3 resource templates and 5 prompts. A connector host reaches **76.4 per cent** of that and cannot list, read, subscribe to or invoke the rest — which is a real containment and a real limitation, and which pushes server authors targeting connector clients back towards publishing context as tools. Templates are the primitive that changes the arithmetic: one published `git://repo/{ref}/{path}` stands for every file at every revision, so the length of `resources/list` says very little about what a server can show you.

Reading results correctly means reading two channels. A **protocol error** replaces the result; a **tool execution error** *is* a successful result carrying `isError: true`. Over eighteen fixtures, a reader that checks the JSON-RPC envelope scored **12 of 18 and read all six execution errors as successes**; a reader that checks `isError` scored 12 and raised on all six protocol errors; a reader that checks both scored **18 of 18**, and it is four lines longer than either. The consequence is not only monitoring: the specification says execution errors **SHOULD** go to the model because they let it self-correct, so a client that cannot tell the channels apart either feeds the model noise or tells it a failed call succeeded.

Two more asymmetries close the chapter. `outputSchema` places a **MUST** on the server and a **SHOULD** on the client, and nothing in between checks: four of six BUILT results did not conform and a non-validating client accepted all six — while the one carrying an undeclared field *did* conform, because `additionalProperties` is absent and Chapter 23's habits do not transfer. And tool annotations are, by specification, **untrusted**: a gate that believes them passed three of eight tools including one that refunds money and one that shares a calendar publicly, against one for a gate derived from the catalogue in Chapter 34's shape. Two more effectful tools declared nothing at all, which is the harder case, because silence needs a default and every default is wrong for half the catalogue. Resource annotations are the honourable exception — `audience`, `priority` and `lastModified` have defined meanings, and using them cut a reading budget by **34.0 per cent** with no judgement calls.

Finally, pagination, which fails more quietly than anything else here. All four list operations paginate with an opaque cursor whose page size the server chooses and the client **MUST NOT** assume. A one-shot client missed **5 of 55 tools** at page size 50 and **54 of 72 items** at page size 5, with no error on either side — the server offered a `nextCursor` and the client did not ask.

KEY TAKEAWAYS

1. Three primitives, three controllers: tools are model-controlled, resources application-driven, prompts user-controlled. **A primitive is a decision about who decides.**
2. Context behind a tool will be fetched whenever the topic looks close enough. Make it a resource, or a narrow tool that returns a fact rather than a file.
3. A resource is identified by a URI; a template publishes a shape rather than an instance, and lives at its own list method.
4. `subscribe` and `listChanged` are independent, and a server may support neither, either or both. Check before designing around them.
5. A missing resource is a **protocol error**, `-32002` — there is no result to flag.
6. `isError: true` lives inside a **successful** result. An envelope-only reader scored 12 of 18 and called every execution error a success.
7. Execution errors SHOULD go to the model; protocol errors MAY. Only one of them is something a model can act on.
8. `outputSchema` is a MUST on the server and a SHOULD on the client. Undeclared fields are permitted unless the schema says otherwise — Chapter 23's rule does not apply.
9. **Tool annotations are untrusted by specification**, and silence is not safety. Derive the safety decision at your call site.
10. All four list operations paginate, page size is the server's, and a one-shot client silently missed 54 of 72 items.

39.36 Glossary

Term	Meaning
Resource	Context data identified by a URI, listed with `resources/list` and fetched with `resources/read`. Application-driven.
Resource template	An RFC 6570 URI template published at `resources/templates/list`, standing for an unbounded set of resources.
Tool	A model-controlled operation, listed with `tools/list` and invoked with `tools/call`.
Prompt	A user-controlled named template returning messages, listed with `prompts/list` and fetched with `prompts/get`.
`audience`	A resource annotation: `"user"`, `"assistant"` or both.
`priority`	A resource annotation from 0.0 to 1.0; 1 is "effectively required", 0 "entirely optional".
`lastModified`	A resource annotation carrying an ISO 8601 timestamp.
`subscribe`	A `resources` sub-capability permitting `resources/subscribe` and `notifications/resources/updated`.
`listChanged`	A sub-capability on tools, resources or prompts announcing that the *set* has changed.
Protocol error	A JSON-RPC `error` object replacing the result: unknown tool, malformed request, server error.

Term	Meaning
Tool execution error	A successful result carrying `isError: true` and content explaining the failure.
`structuredContent`	Server-produced JSON in a tool result. Unrelated to constrained decoding.
`outputSchema`	An optional JSON Schema for `structuredContent`. Servers MUST conform; clients SHOULD validate.
`resource_link`	A tool result content type pointing at a resource URI without embedding it. Not guaranteed to appear in `resources/list`.
Embedded resource	A content block of type `resource` carrying a full resource inline, usable in tool results and prompt messages.
`execution.taskSupport`	A tool field with values `forbidden` (default), `optional` or `required`.
Cursor	An opaque pagination token. Never parsed, never modified, never persisted across sessions.
`nextCursor`	The field whose absence means the list has ended.

39.37 Further Reading

Primary sources. Specification revision **2025-11-25** at `modelcontextprotocol.io/specification/2025-11-25`: `server/resources` for URIs, templates, subscriptions and annotations, `server/tools` for the tool object, both error channels, `outputSchema` and the untrusted-annotations warning, `server/prompts` for arguments and embedded resources, and `server/utilities/pagination` for the cursor rules. The `basic` page carries the JSON Schema dialect rules that govern both `inputSchema` and `outputSchema`. `server/utilities/completion` covers argument auto-completion for prompts and resource templates, which Chapter 40 takes up.

Where to go next in this book. Chapter 40 closes Part 7 by building a server: the authorization framework, the completion and logging utilities, packaging, and the attacks from Chapter 38's security section seen from the other side of the boundary. **Chapter 41** opens Part 8 with Claude Code, where resources appear as `@server:protocol://path` mentions and prompts as slash commands — the two primitives this chapter argued are the ones people forget. For choosing what enters a limited context window, **Chapter 52**; for retrieval over a large resource set, **Chapter 53**; for what a hostile resource can do once it is in the window, **Chapter 58**.